
Capturing non-Markovian effects in weather prediction models

Abstract

1 Introduction to the architecture

Turbulent flows are known as a notoriously difficult framework for numerical simulations. One of the common method in such simulations is the *direct numerical simulations* (DNS) that discretize the Navier-Stokes equation to a space and time grid and then simulate directly. Such discretizations, however require a very fine grid resolution scaling as $N_{\text{grid}} \sim (Re)^\alpha$ with $\alpha \sim 2.05$ Yang and Griffin [2021]. Insufficient small-scale resolution may leave mean dissipation and enstrophy approximately converged while substantially underestimating high-order moments of velocity gradients, dissipation, and enstrophy, as well as the tails of velocity-gradient distributions and statistics involving second velocity derivatives Donzis et al. [2008]. While this becomes expensive, a variety of models that regularize the equations on a larger scale, introducing proper dissipative terms at the cost of short time accuracy Wilson and Sawford [1996]. Another approach consists in keeping coarse-grained scales, without non-controlled dissipative approximation, but at a cost of keeping the non-Markovian memory terms Lin et al. [2021], De Wit et al. [2026].

In this article, we explore the potential of such methods for a weather prediction models.

1.1 Two Hybrid architectures

For our study, we select a Markovian pretrained model, and intentionally learn a non-Markovian kernel for a temporal interaction. In this article, we introduce and study a hybrid GraphCast-Mamba architecture, that captures non-Markovian effects. This architecture, is shown schematically in Fig. 1b.

GraphCast For a baseline Markovian model we use a DeepMind checkpoint of the GraphCast model. This is a graph neural network (GNN) model which operates on two grids - original latitude-longitude grid, and the auxiliary mesh grid. The full model operates on a graph $\mathcal{G}(\mathcal{V}^G, \mathcal{V}^M, \mathcal{E}^{G2M}, \mathcal{E}^M, \mathcal{E}^{M2G})$, where $\mathcal{V}^G$ is the original latitude-longitude grid containing the physical data, $\mathcal{V}^M$ is a much smaller mesh grid. The model first a first mapping $\mathcal{E}^{M2G}$ from grid-to-mesh, then N ($N = 16$ for resolution 0.25° GraphCast model) processor message passing steps $\mathcal{E}^M$ on a mesh grid and finally grid-to-mesh mapping $\mathcal{E}^{G2M}$ back to the original grid. Each node has the same dimension W_{GC} (canonical GC value $W_{GC} = 512$). The total number of mesh nodes N_{mesh} is typically much smaller than the number of grid nodes N_{grid} .

Mamba Mamba is an attention-free sequence model based on selective state-space models (SSMs), designed to model long temporal dependencies with computation that scales linearly with sequence length Gu and Dao [2023]. A conventional continuous-time SSM maps an input sequence $x(t)$ to a latent memory state $h(t)$ according to

$$\frac{dh(t)}{dt} = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t). \quad (1)$$

Here, A is a diagonal matrix that determines how quickly memory decays or evolves, B and C determines how new inputs are written into the state and readout from the state respectively, and D provides a direct input-to-output skip connection. For a single SSM channel, each of the matrices have fixed dimension $d_{\text{state}} = 16$ and Δ - is a scalar. If A , B , and C are fixed, the same linear dynamical system is applied at every time step. For a discrete step size Δ , its exact discretization is

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad \bar{A} = \exp(\Delta A), \quad \bar{B} = A^{-1}[\exp(\Delta A) - I]B. \quad (2)$$

Mamba makes this SSM *selective*: rather than using fixed input and readout matrices and a fixed step size, it computes B_t , C_t , and Δ_t from the current input. Thus, the model can decide which information to retain, overwrite, or expose at each time step.

In our model, Mamba operates independently on the temporal sequence of each GraphCast mesh node. Let $x_t^{\text{in}} \in \mathbb{R}^{W_{\text{GC}}}$ denote the mesh latent feature at a given node and forecast step. A layer-normalized input is projected into two d_{inner} -dimensional branches (typically $d_{\text{inner}} = W_{\text{GC}}/4$),

$$[\tilde{x}_t, z_t] = W_{\text{in}} \text{LN}(x_t^{\text{in}}), \quad \tilde{x}_t, z_t \in \mathbb{R}^{d_{\text{inner}}}. \quad (3)$$

The first branch, $\tilde{x}_t$, supplies the SSM input. The second branch, z_t , is a learned output gate: it does not update the memory state directly, but controls how much of the SSM output is passed onward.

Before entering the SSM, $\tilde{x}_t$ is processed by a depthwise causal one-dimensional convolution with kernel width four:

$$u_{t,d} = \text{SiLU} \left(b_d^{\text{conv}} + \sum_{k=0}^{d_{\text{conv}}-1} w_{d,k}^{\text{conv}} \tilde{x}_{t-k,d} \right), \quad d = 1, \dots, d_{\text{inner}}, \quad (4)$$

where $\text{SiLU}(a) = a/(1 + e^{-a})$. ‘‘Depthwise’’ means that each of the d_{inner} channels has its own d_{conv} learned convolution weights $w_{d,k}^{\text{conv}}$ and does not mix with other channels at this stage. ‘‘Causal’’ means that u_t depends only on the current input and the three previous inputs, never on future inputs. During autoregressive inference, the preceding three projected inputs are retained in a convolution cache, so this local temporal context continues across forecast calls.

The selective parameters are generated from u_t by learned linear projections:

$$[r_t, B_t, C_t] = W_x u_t, \quad r_t \in \mathbb{R}^{d_t}, \quad B_t, C_t \in \mathbb{R}^{d_{\text{state}}}, \quad (5)$$

$$\Delta_t = \text{softplus}(W_{\Delta} r_t + b_{\Delta}) \in \mathbb{R}^{d_{\text{inner}}}. \quad (6)$$

($d_t = 32$, $d_{\text{state}} = 16$) All matrices, vectors, convolution kernels, and biases in these equations are trainable parameters optimized end-to-end by gradient descent using the forecasting loss. In contrast, B_t , C_t , and Δ_t are not fixed parameters: they are input-dependent values generated anew at each time step from u_t . The softplus function ensures $\Delta_t > 0$.

The continuous-time state matrix is diagonal and parameterized as

$$A_{d,n} = -\exp(A_{\log,d,n}), \quad d = 1, \dots, d_{\text{inner}}, \quad n = 1, \dots, d_{\text{state}}. \quad (7)$$

Because every element of A is negative, the corresponding factor $\exp(\Delta_{t,d} A_{d,n})$ lies between zero and one, giving stable, decaying memory. The implementation uses the following selective recurrence:

$$h_{t,d,n} = \exp(\Delta_{t,d} A_{d,n}) h_{t-1,d,n} + \Delta_{t,d} B_{t,n} u_{t,d}. \quad (8)$$

The first term retains a scaled portion of the previous state, while the second writes the current input into memory. A small $\Delta_{t,d}$ gives slow decay and long memory; a large $\Delta_{t,d}$ gives faster decay and a stronger update from the current input.

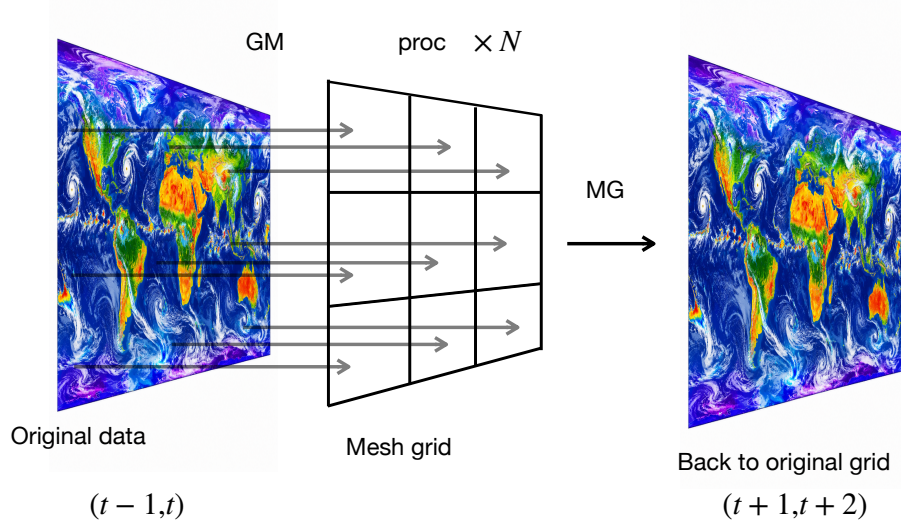
The state is read out using the selective vector C_t and a learned skip parameter D :

$$s_{t,d} = \sum_{n=1}^{16} C_{t,n} h_{t,d,n} + D_d u_{t,d}. \quad (9)$$

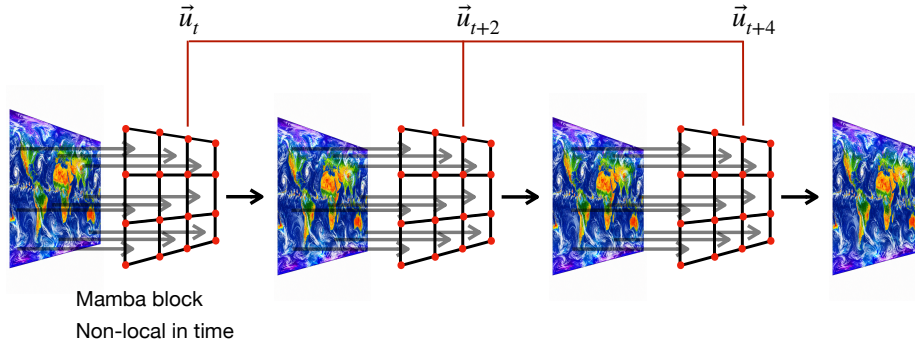
Finally, the gate branch z_t modulates this readout, and the result is projected back to the W_{GC} -dimensional GraphCast latent space:

$$o_t = W_{\text{out}} [s_t \odot \text{SiLU}(z_t)], \quad x_t^{\text{out}} = x_t^{\text{in}} + o_t. \quad (10)$$

Our main configuration uses an internal channel dimension of 128, an SSM state dimension of 16 per channel, a causal convolution width of 4, and a rank-32 projection for Δ_t . The SSM state h_t and the three-token convolution cache persist separately for every batch element, mesh node, Mamba layer, and autoregressive forecast step.



(a) Schematic GraphCast architecture, original data are projected to a mesh grid, and after the iteration of N processor steps, projected back to the original grid.



(b) GraphCast-Mamba architecture, we additionally insert blocks of Mamba neural network, with a hidden state represented by a red dot. The dynamics of the hidden state, simulates the non-Markovian connection.

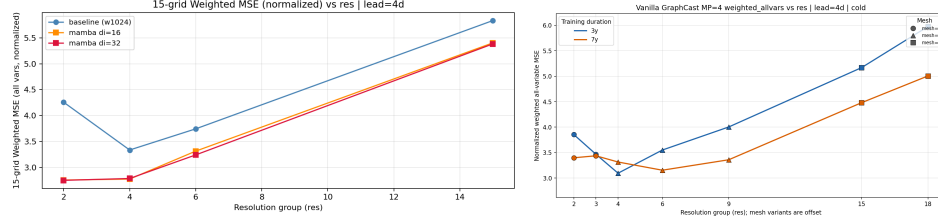


Figure 2: Comparison of models with different resolutions and the size of training dataset.

Additive and residual Mamba branches. We investigate two ways of combining Mamba with a pretrained GraphCast forecast. Let $G_\theta(X_t)$ denote the prediction of the pretrained GraphCast baseline from the meteorological input history X_t . In both cases, GraphCast provides a strong physical-spatial forecast, while Mamba provides temporally persistent information through its recurrent state. The final prediction is expressed as the baseline forecast plus a learned correction.

In the *additive Mamba* formulation, Mamba is a separate correction branch, such that the final prediction $\hat{Y}_t = G_\theta(X_t) + c_t^{\text{add}}$ is a linear combination of the GraphCast prediction and a shallow GraphCastMamba model trained separately.

$$c_t^{\text{add}} = M_\phi(Z_t; h_{t-1}), \quad h_t = \text{MambaStateUpdate}_\phi(Z_t, h_{t-1}), \quad (11)$$

Here, h_t is the persistent Mamba state, and ϕ denotes the trainable parameters of the Mamba branch. The baseline GraphCast model can remain frozen, so optimization learns only the additive correction. This construction is modular: GraphCast generates the main forecast, while Mamba learns the temporal component of the error that remains after the baseline prediction.

In the *residual Mamba* formulation, Mamba is instead integrated into a trainable residual GraphCast head. The pretrained GraphCast backbone first provides mesh latent features L_t , and a residual graph neural network (GNN) further processes these features. Mamba blocks are inserted between GNN message-passing updates, allowing temporal memory to directly modify the evolving mesh representation:

$$L_t^{(0)} = L_t, \quad \tilde{L}_t^{(k)} = \text{GNN}_\phi^{(k)}(L_t^{(k-1)}), \quad (12)$$

$$L_t^{(k)} = \tilde{L}_t^{(k)} + \text{Mamba}_\phi^{(k)}(\tilde{L}_t^{(k)}; h_{t-1}^{(k)}), \quad k = 1, \dots, K. \quad (13)$$

The resulting features are decoded into a residual forecast,

$$c_t^{\text{res}} = \text{Decode}_\phi(L_t^{(K)}), \quad \hat{Y}_t = G_\theta(X_t) + c_t^{\text{res}}. \quad (14)$$

The essential difference is therefore where the temporal model acts. Additive Mamba predicts a correction in a separate branch and combines it with GraphCast only at the output. Residual Mamba injects the Mamba update into the residual GNN latent representation, so temporal memory can interact with graph message passing before the correction is decoded. In both approaches, the Mamba state is carried forward across autoregressive forecast steps, enabling the correction to depend on information accumulated over the preceding forecast trajectory.

2 Numerical results

2.1 Mamba vs resolution

Initially, we noted a non-monotonic dependence of the model accuracy, depending on the initial grid resolution. However, later we observed that the shape of this curve is highly influenced by the amount of training data see Fig. 2. Interestingly, addition of Mamba branch improves the high resolution models, an probably demands less data to learn.

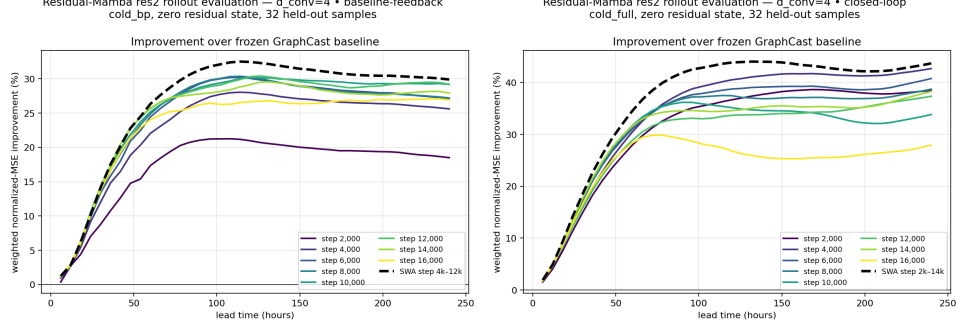


Figure 3: Additive Mamba branch, a comparison of open loop model, and closed loop, both with and without SWA. $d_{inner} = 16, d_{state} = 16$

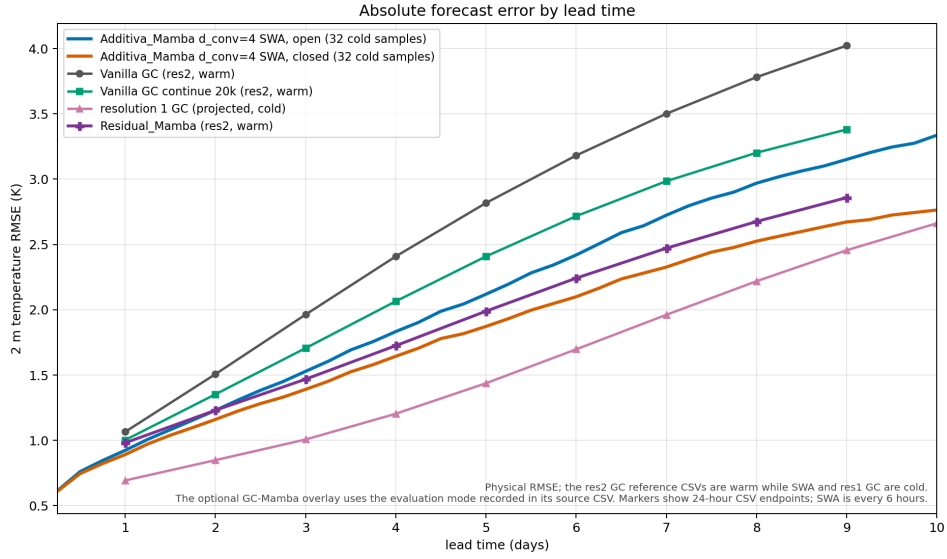


Figure 4: Comparison of residual and additive Mamba models, the much larger DeepMind checkpoint GraphCast small is provided for comparison

2.2 Small resolution models

First, we perform a study of resolution 2° models. Here, we pretrain the standard small sized GraphCast models, with only 6 message passing steps, and width $W = 512$. On top of it, we learn both the additive and residual Mamba blocks. During this training, we optimize the error at $K = 12$ timesteps, which is equal to a 4 day forecast that is generated autoregressively (AR) by feeding the previous output to the next input, with continuous updates of the mamba state.

We explored two possibilities of this AR unrolling: the so-called open and closed loops. In the open loop formulation we don't feed the full prediction to the next step, but only a part generated by the baseline GraphCast model – this enforces model to be closer to an original GraphCast, and prevents instabilities related to an unphysical grow of Mamba part. The other strategy is the closed loop, which feeds back the full prediction. We also noted, that, especially for closed loop, eval loss behaves quite randomly, this is a signal that additional use of SWA technique, might help a lot. The results are provided in Fig. 3.

Similarly, we trained the residual Mamba model, with the closed loop only. The comparison of the two models are plotted in Fig. 4

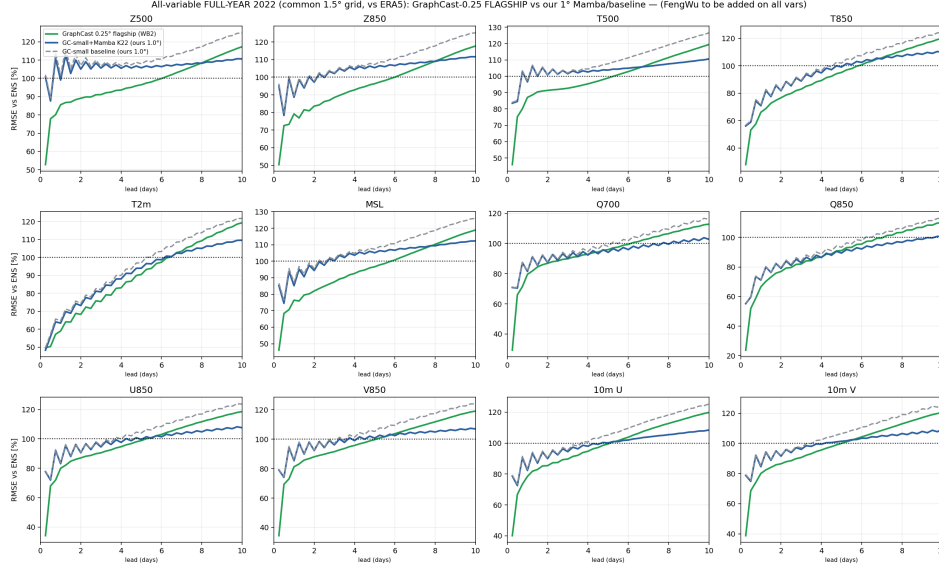


Figure 5: Comparison of the GraphCast small (resolution 1°) model, and the additive Mamba model trained on top of it. The Results of GraphCast large (0.25°) are shown for comparison.

2.3 resolution 1°

For the resolution 1° model, we take the DeepMind trained GraphCast small checkpoint, and train the additive Mamba branch. The numerical result is provided in Fig. 5. We see that the addition of an additive Mamba branch can significantly improve the long-time accuracy, without compromising short times.

3 ToDo:

3.1 Why larger Mamba models do not give any improvement?

3.2 Compare residual and additive branch more for resolution 2° and also resolution 1°

3.3 Train GraphCast large (0.25° resolution)

References

- Xander M De Wit, Alessandro Gabbana, Michael Woodward, Yen Ting Lin, Federico Toschi, and Daniel Livescu. Data-driven mori–zwanzig modeling of lagrangian particle dynamics in turbulent flows. *Proceedings of the National Academy of Sciences*, 123(13):e2525390123, 2026.
- D. A. Donzis, P. K. Yeung, and K. R. Sreenivasan. Dissipation and enstrophy in isotropic turbulence: Resolution effects and scaling in direct numerical simulations. *Physics of Fluids*, 20(4):045108, 04 2008. ISSN 1070-6631. doi: 10.1063/1.2907227. URL <https://doi.org/10.1063/1.2907227>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization, 2019. URL <https://arxiv.org/abs/1803.05407>.
- Yen Ting Lin, Yifeng Tian, Daniel Livescu, and Marian Anghel. Data-driven learning for the mori–zwanzig formalism: A generalization of the koopman learning framework. *SIAM Journal on Applied Dynamical Systems*, 20(4):2558–2601, 2021. doi: 10.1137/21M1401759. URL <https://doi.org/10.1137/21M1401759>.

John D. Wilson and Brian L. Sawford. Review of lagrangian stochastic models for trajectories in the turbulent atmosphere. *Boundary-Layer Meteorology*, 78(1):191–210, 1996. ISSN 1573-1472. doi: 10.1007/BF00122492. URL <https://doi.org/10.1007/BF00122492>.

Xiang IA Yang and Kevin P Griffin. Grid-point and time-step requirements for direct numerical simulation and large-eddy simulation. *Physics of Fluids*, 33(1), 2021.

A Resolution Projection

B Stochastic Weighted Average

ref Izmailov et al. [2019]